2026 Distributed and Parallel Computing

Logical Time, Causality, and Lamport's Clocks

Hongik University, Department of Software and Communications Engineering.

Lan Anh, Nguyen

L. Lamport, "Time, Clocks, and the Ordering of Events in a Distributed System," *Communications of the ACM, vol. 21, no. 7, pp. 558–565, July 1978. DOI: 10.1145/359545.359563.*

Contents

1. Time, Clock, Order in Distributed Systems?
2. Lamport's Clock

Clocks in Distributed Systems

- **Time** is *crucial* in distributed systems, used everywhere:
  - Ordering events
  - Resolving data conflicts
  - Setting timeout
  - Expiring caches
  - etc.

- Clocks a fundamental timing mechanism that generates regular, precisely timed electrical signals
  - Each node has its own clock

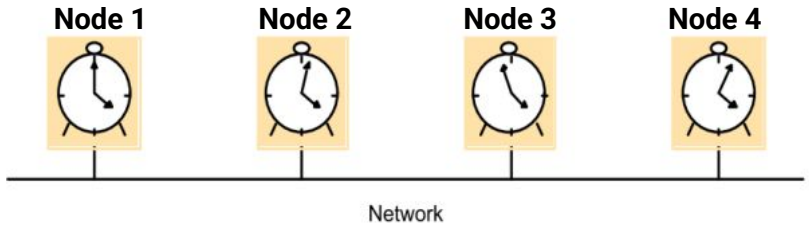
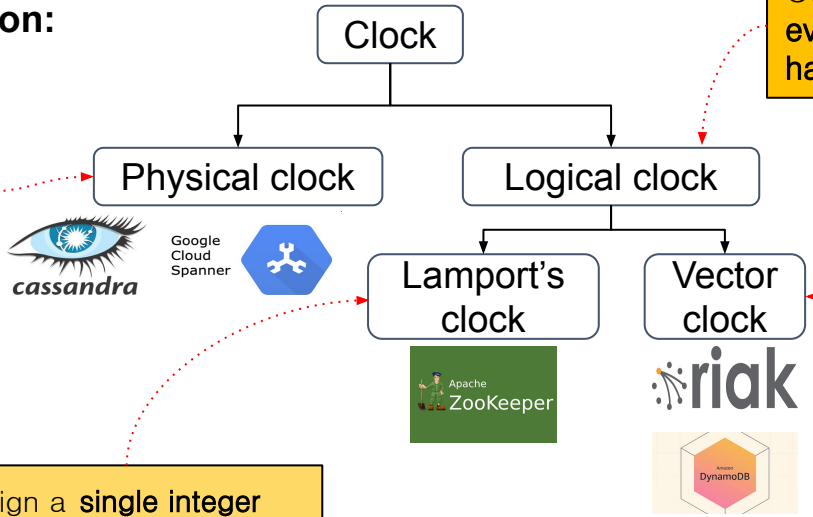


Fig 1. Clock in distributed system

Clocks in Distributed Systems

- **Clock classification:**

Corresponds to **actual time** in real-world (hours, minutes, seconds, etc.).
Focuses on **"When happened"**.



Corresponds to **ordering of events.** Focuses on **"What happened before what".**

Assign a **single integer** to each event to preserve causal ordering.
Can not identify concurrent events.

Assign **an integer vector of counters** to each event to track causal ordering.
Can identify concurrent events.

We focus on Lamport's clock today!!!

L. Lamport, "Time, Clocks, and the Ordering of Events in a Distributed System," Communications of the ACM, vol. 21, no. 7, pp. 558–565, July 1978. DOI: 10.1145/359545.359563.

Clocks in Distributed Systems

- **Physical clocks:** may not preserve correct ordering, because **deal with problems**:
  - Skew: Difference between 2 clocks at one point in time
  - Drift: Two clocks tick at different rates
    - Reasons: Different frequencies, plus heat, humidity, voltage, etc.

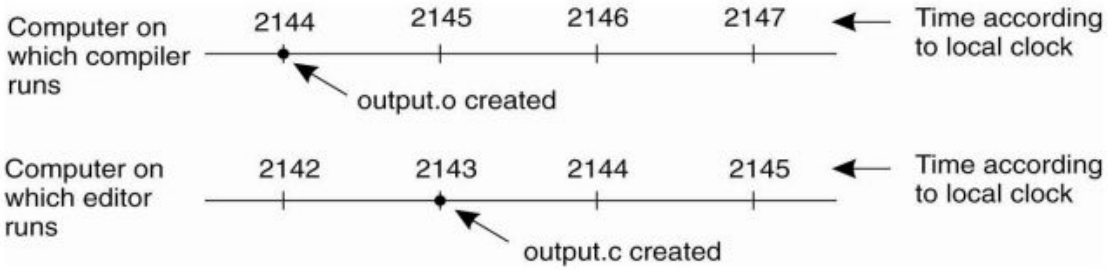


Fig 2. Wrong ordering in physical clocks (Wrong ordering forces computer not to recompile new edits)

Clocks in Distributed Systems

- **Physical clocks** use **clock synchronization mechanisms** to overcome problems
  - Network time protocol
  - Cristian's algorithm
  - Berkeley algorithm
  - etc.

- Eventhough **clock synchronization can solve problems** of physical clocks, there are still some issues:
  - Not perfect, when skew and drift still exist
  - Can not identify causal ordering
  - Sync delay
  - Some cases, absolute time is not important:
    - Nodes do not interact, no need to synchronize them
    - Nodes agree on the order in which events occur rather than the time

Logical clock (Lamport's clock and vector clock) addresses issues

Event Ordering in Logical Clocks

- Problem a logical clock needs to solve:
  Define a global ordering of all events in distributed system based on "Happened-Before Relation", whereas:
  - The logical clock assigns a logical number (integer) for each event, timestamp
    - E.g.,

$$C_i(a) = \text{logical time assigned to event } a \text{ at process } P_i$$

  - No global clock, local clocks may be unsynchronized

- **Lamport's key ideas**: Use happened-before relation
  - Processes of nodes exchange messages
  - Message must be sent before received
  - Send/receive used to order events

Happened-Before Relation (HB)

- It defines event a happened before event b:

$$a \rightarrow b$$

- Based on 3 rules:
  - ***On same process:*** a occurs before b
  - **Send/receive message between different processes:** a sends, b receives
  - **Transitivity:** $a \rightarrow b \text{ and } b \rightarrow c \Rightarrow a \rightarrow c$

- HB gives **"partial ordering":** Only causally related event pairs are ordered
  - Unordered events are concurrent
  - When event a and b are concurrent, $a \parallel b$, we could not order them like $a \rightarrow b$ $b \rightarrow a$

Happened-Before Relation (HB)

- Example:

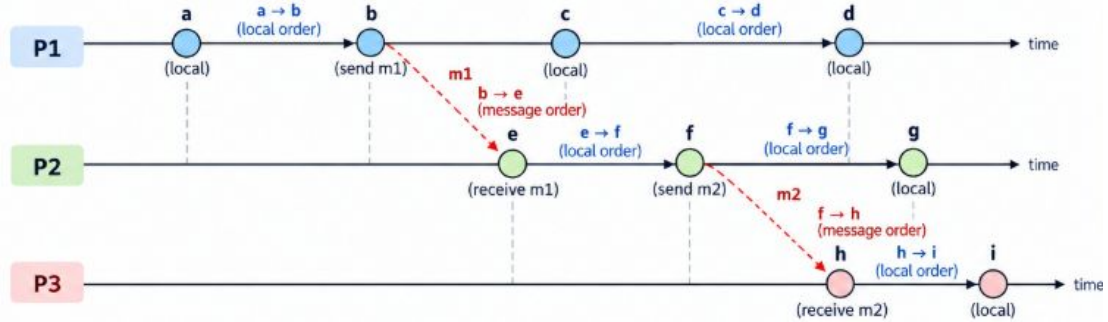


Fig 3. Happened-before and concurrent events

Happened-before:
a->b; b->c; b->e; b->f; …
Concurrent events:
c//f; g//i;

Lamport'c Clocks Condition

- Event **a happened before event b**, then **timestamp(a) < timestamp(b)**
  - **Logical timestamp** is numeral counter assigned to an event
  - Equation:

$$a \rightarrow b \Rightarrow C(a) < C(b)$$

Standard Lamport's algorithm

1. Each process P_i maintains a logical clock (C) with logical timestamp C_i
2. Whenever an event occurs locally at P_i, update timestamp $C_i = C_i + 1$
3. When P_i sends a message to P_j, timestamp which message brings is $ts(m) = C_i$
4. When P_j receives message, logical timestamp of process j can be updated to:

$$C_j \leftarrow \max(C_j, ts(m) + 1)$$

=> Together they guarantee the clock condition.

Lamport's algorithm

- Examples: Standard Lamport's clock

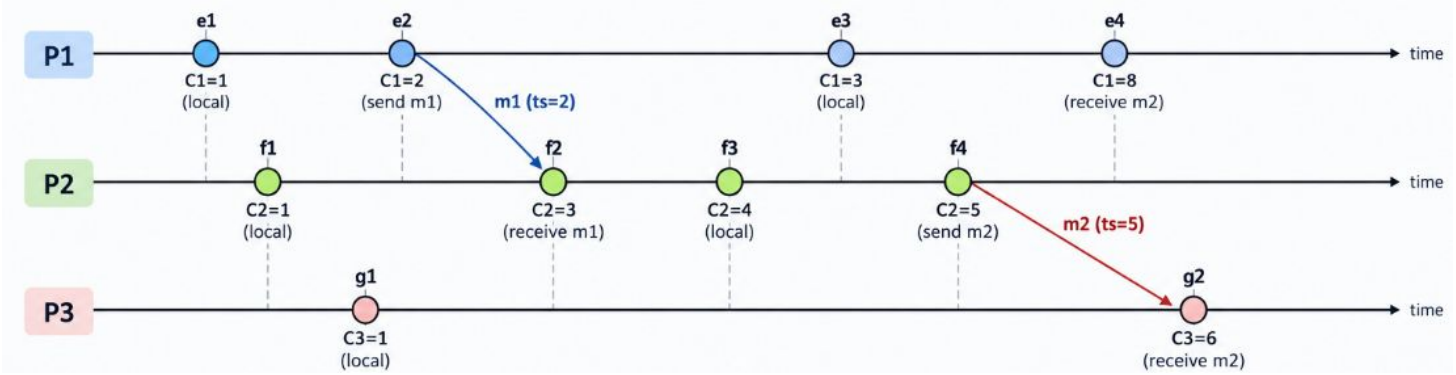


Fig 4. Lamport's clocks

On P1: C1(e2) = 2;
On P2: C2(f2) = Max(C2(f1), m1+1) = 2 + 1 = 3;
On P3: C3(g2) = Max(C3(g1), m2+1) = 5 + 1 = 6;

Lamport's algorithm

- Examples: Use Lamport's clock to adjust timestamps of clocks
  - 3 clocks at different rates: 6, 8, 10
  - Only use Lamport's clock to adjust timestamp with wrong clock condition: Ci = Max(Ci, Mess + d), d=1

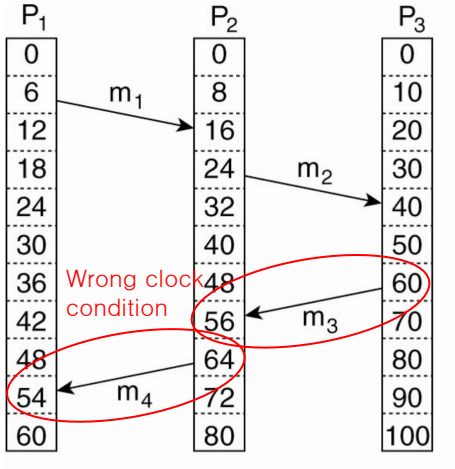


Fig 5-a. Normal clocks without Lamport's clock

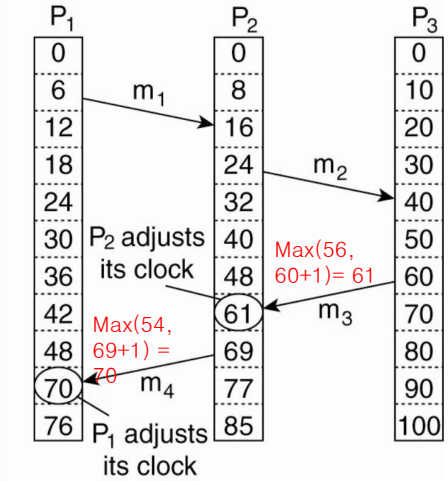


Fig 5-b. Use Lamport's clocks to adjust clocks

Limitation of Lamport's Clock (1): Causality not fully revealed

- **Causality** describes the relationship between events in which one event could have influenced or affected another event.
- With Lamport's clocks:
  - We have: $a \rightarrow b \Rightarrow C(a) < C(b)$
  - **Problem: Reverse is not true or does not fully reveal causality**
    - Means that nothing about events can be inferred from timestamp comparison
    - **Solution:** Add a process ID (PID) to message

C(c) < C(d): c and d are concurrent ~ no causality between them

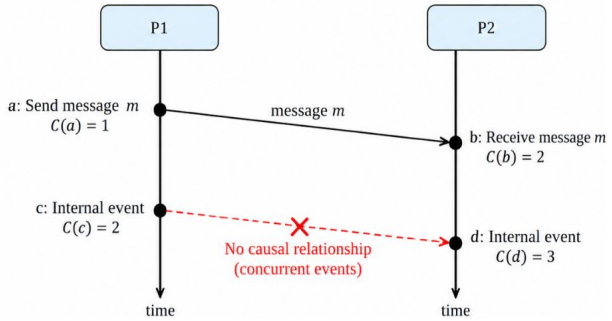


Fig 6. Causality not fully revealed

Limitation of Lamport's Clock (2): No physical time

- **Physical time** refers to real elapsed time between events (E.g., seconds)
- With Lamport's clocks, we just know logical timestamps, providing ordering
  - **Problem: No real-time information**
    - Can not determine timeout or whether a process fails using logical timestamps alone
    - **Solution:** Use physical clocks

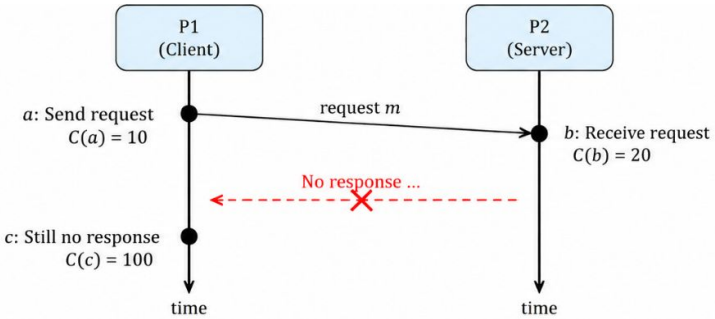


Fig 7. No physical time

C(a) = 10; C(c) = 100;
"C(c) − C(a) = 90" does not mean 90 seconds elapsed.

Using logical timestamps, P1 can not determine whether P2 has failed or slowly

Limitation of Lamport's Clock (3): Hidden external causality

- **External causality** describes causal relationships created by events outside the distributed system
- With Lamport's clocks, causality is captured only from local events inside the system
  - **Problem: External causal relationships may be invisible to the system**
    - A later event may receive a smaller timestamp even though it is caused by an earlier event
    - **Solution:** Pass timestamp, physical clocks

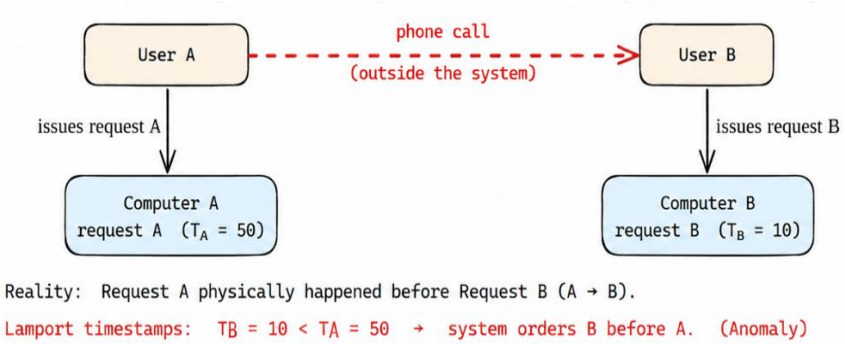


Fig 8. Hidden external causality

System event a: User A issues request A
External event: User A calls User B
System event b: User B issues request B

T(A) = 50 > T(B) = 10 -> B before A (x)
Actually A happened before B

Q&A
Thank You